Exemple 01 — Monter le lab de cybersécurité et vérifier la connectivité

Le coeur du LAB : le fichier docker-compose.yml

```
services:
  dvwa-cible:
    image: vulnerables/web-dvwa
    container_name: dvwa-cible
    ports:
      - "127.0.0.1:8088:80"
    networks:
      cyberlab-net:
        ipv4_address: 10.40.0.10

  attaquant:
    image: debian:12
    container_name: attaquant
    cap_add:
      - NET_ADMIN
      - NET_RAW
    command: sleep infinity
    volumes:
      - ./:/data
    networks:
      cyberlab-net:
        ipv4_address: 10.40.0.20

networks:
  cyberlab-net:
    driver: bridge
    ipam:
      config:
        - subnet: 10.40.0.0/16
```

Déploiement du LAB dur une VM Debian avec Docker :

→ Lancement des conteneurs

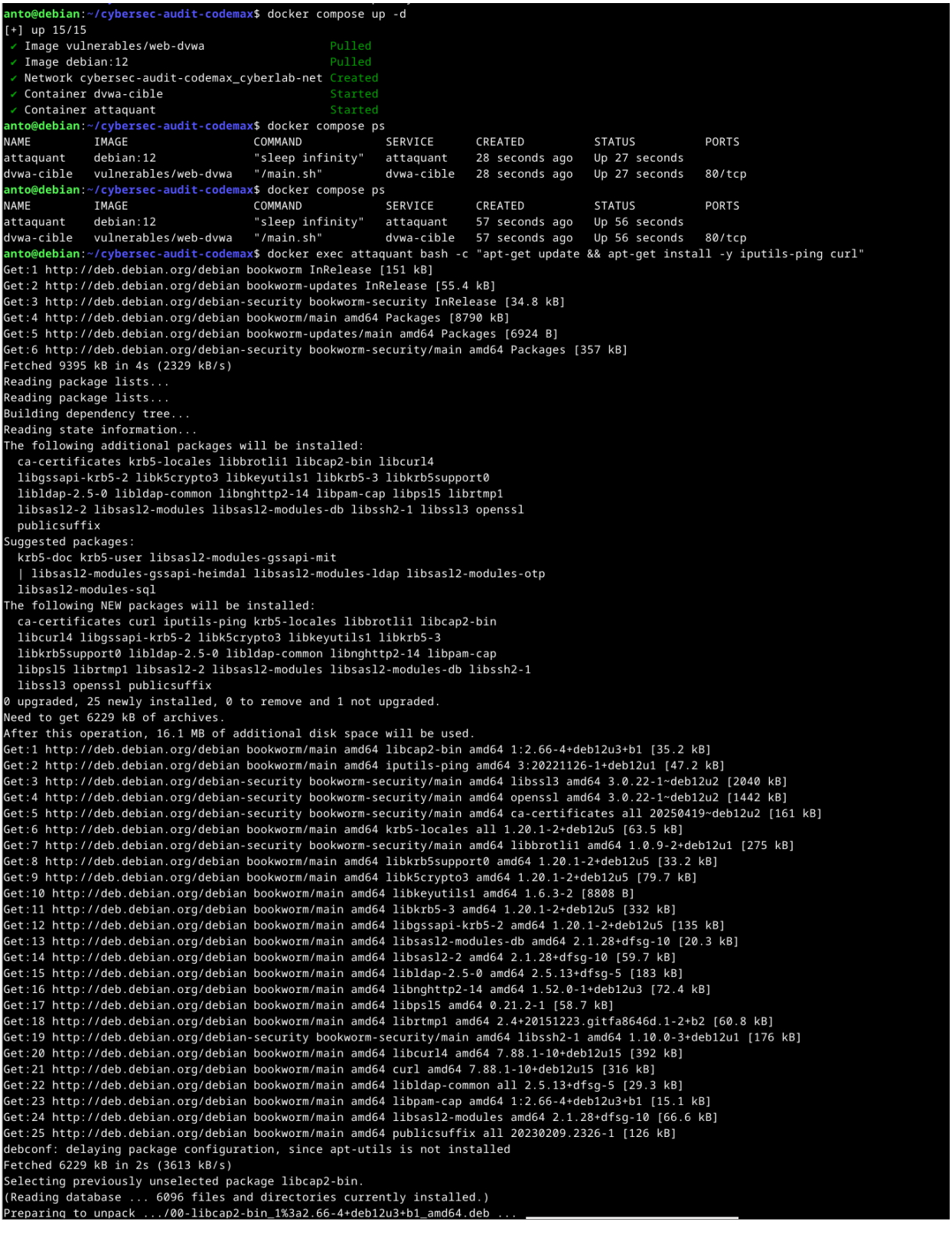
docker compose up -d

→ Vérification de leur état

docker compose ps

→ Installation des outils réseau dans l'attaquant

docker exec attaquant bash -c "apt-get update && apt-get install -y iputils-ping curl"



→ Prouver que l'attaquant atteint la cible

docker exec attaquant ping -c 3 10.40.0.10

→ Confirmer que le service web DVWA répond

docker exec attaquant curl -s -o /dev/null -w "%{http_code}\n" http://10.40.0.10/

→ Initialiser la base de données de DVWA

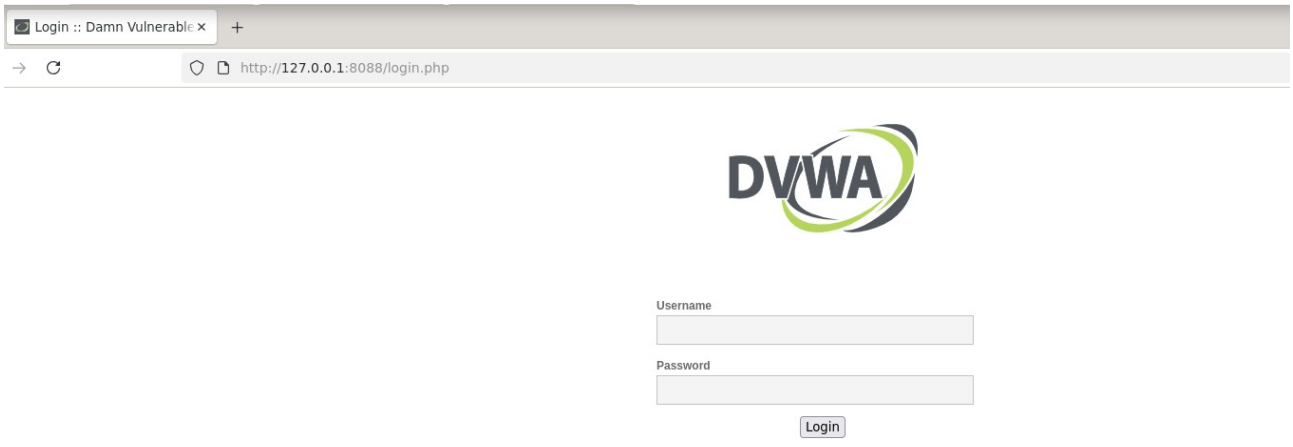
docker exec attaquant sh -c "rm -f /tmp/ck; curl -s -c /tmp/ck http://10.40.0.10/setup.php | grep -oE '[a-f0-9]{32}' | head -1 | xargs -I TKN curl -s -b /tmp/ck --data create_db=1 --data user_token=TKN http://10.40.0.10/setup.php -o /dev/null; echo BASE_DVWA_PRETE"

```
anto@debian:~/cybersec-audit-codemax$ docker exec attaquant ping -c 3 10.40.0.10
PING 10.40.0.10 (10.40.0.10) 56(84) bytes of data.
64 bytes from 10.40.0.10: icmp_seq=1 ttl=64 time=0.097 ms
64 bytes from 10.40.0.10: icmp_seq=2 ttl=64 time=0.077 ms
64 bytes from 10.40.0.10: icmp_seq=3 ttl=64 time=0.046 ms

--- 10.40.0.10 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2049ms
rtt min/avg/max/mdev = 0.046/0.073/0.097/0.021 ms
anto@debian:~/cybersec-audit-codemax$
anto@debian:~/cybersec-audit-codemax$ docker exec attaquant curl -s -o /dev/null -w "%{http_code}\n" http://10.40.0.10/
302
anto@debian:~/cybersec-audit-codemax$ docker exec attaquant sh -c "rm -f /tmp/ck; curl -s -c /tmp/ck http://10.40.0.10/setup.php | grep -oE '[a-f0-9]{32}' | head -1 | xargs -I TKN
 curl -s -b /tmp/ck --data create_db=1 --data user_token=TKN http://10.40.0.10 /setup.php -o /dev/null; echo BASE_DVWA_PRETE"
BASE_DVWA_PRETE
anto@debian:~/cybersec-audit-codemax$
```

Version interface graphique (dans le navigateur)

→ Le service web répond. La page de connexion de DVWA s'affiche



http://127.0.0.1:8088/index.php

DVWA

- Home
- Instructions
- Setup / Reset DB

- Brute Force
- Command Injection
- CSRF
- File Inclusion
- File Upload
- Insecure CAPTCHA
- SQL Injection
- SQL Injection (Blind)
- Weak Session IDs
- XSS (DOM)
- XSS (Reflected)
- XSS (Stored)
- CSP Bypass
- JavaScript

- DVWA Security
- PHP Info
- About

- Logout

Welcome to Damn Vulnerable Web Application!

Damn Vulnerable Web Application (DVWA) is a PHP/MySQL web application that is damn vulnerable. Its main goal is to be an aid for security professionals to test their skills and tools in a legal environment, help web developers better understand the processes of securing web applications and to aid both students & teachers to learn about web application security in a controlled class room environment.

The aim of DVWA is to **practice some of the most common web vulnerabilities**, with **various levels of difficultly**, with a simple straightforward interface.

General Instructions

It is up to the user how they approach DVWA. Either by working through every module at a fixed level, or selecting any module and working up to reach the highest level they can before moving onto the next one. There is not a fixed object to complete a module; however users should feel that they have successfully exploited the system as best as they possible could by using that particular vulnerability.

Please note, there are **both documented and undocumented vulnerability** with this software. This is intentional. You are encouraged to try and discover as many issues as possible.

DVWA also includes a Web Application Firewall (WAF), PHPIDS, which can be enabled at any stage to further increase the difficulty. This will demonstrate how adding another layer of security may block certain malicious actions. Note, there are also various public methods at bypassing these protections (so this can be seen as an extension for more advanced users)!

There is a help button at the bottom of each page, which allows you to view hints & tips for that vulnerability. There are also additional links for further background reading, which relates to that security issue.

WARNING!

Damn Vulnerable Web Application is damn vulnerable! **Do not upload it to your hosting provider's public html folder or any Internet facing servers**, as they will be compromised. It is recommend using a virtual machine (such as VirtualBox or VMware), which is set to NAT networking mode. Inside a guest machine, you can downloading and install XAMPP for the web server and database.

Disclaimer

We do not take responsibility for the way in which any one uses this application (DVWA). We have made the purposes of the application clear and it should not be used maliciously. We have given warnings and taken measures to prevent users from installing DVWA on to live web servers. If your web server is compromised via an installation of DVWA it is not our responsibility it is the responsibility of the person/s who uploaded and installed it.

More Training Resources

DVWA aims to cover the most commonly seen vulnerabilities found in today's web applications. However there are plenty of other issues with web applications. Should you wish to explore any additional attack vectors, or want more difficult challenges, you may wish to look into the following other projects:

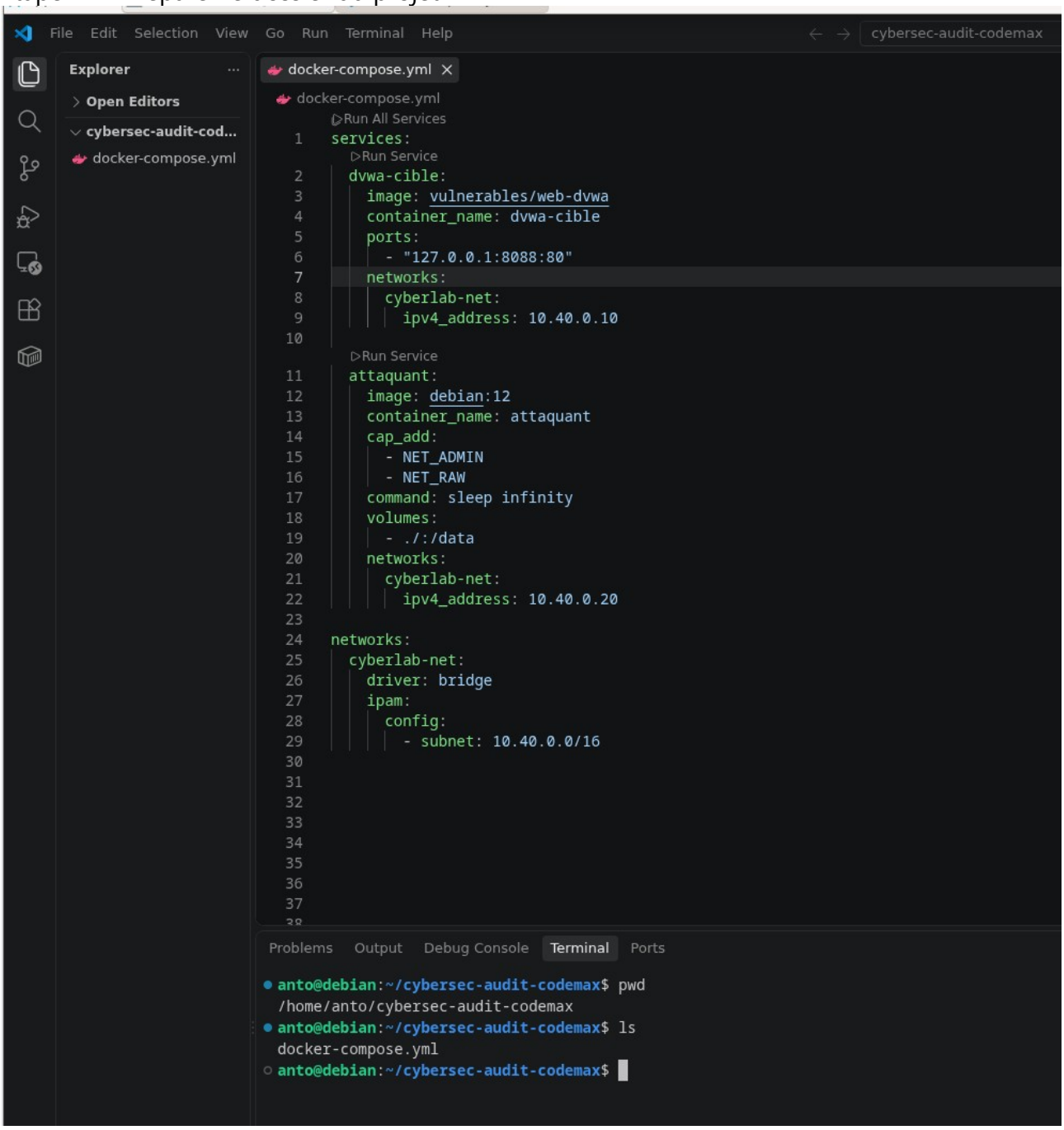
- bWAPP
- NOWASP (formerly known as Mutillidae)
- OWASP Broken Web Applications Project

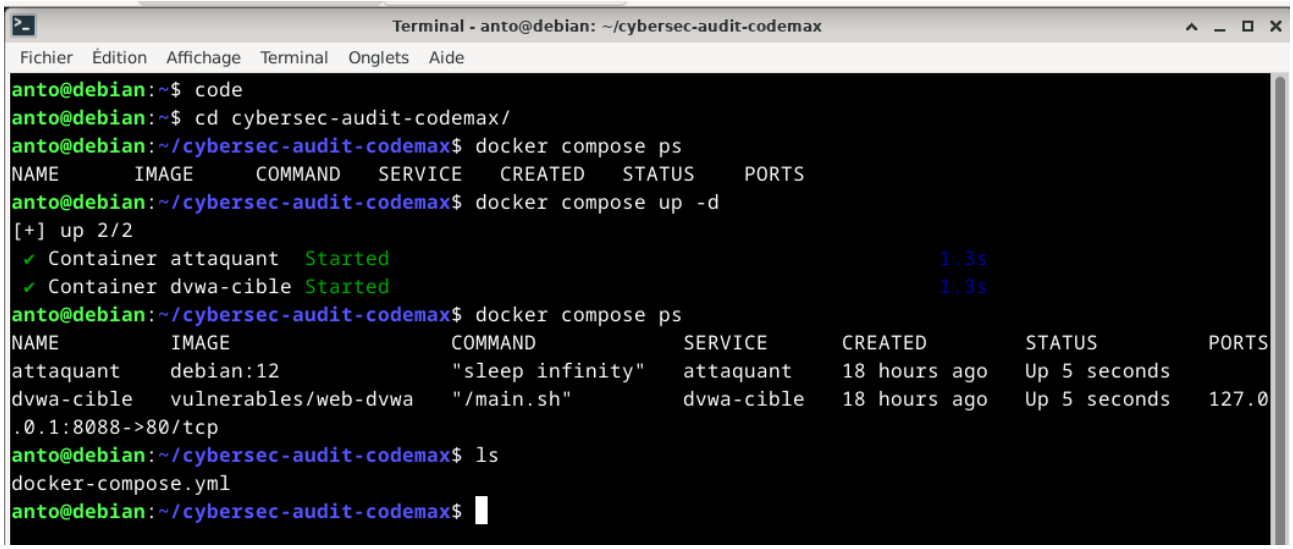
You have logged in as 'admin'

Username: admin
Security Level: low
PHPIDS: disabled

Exercice 02 — Organiser l'audit de Code un maxautour des métiers et des méthodes

Étape 1 — Préparer le dossier du projet





Étape 2 — Rédiger la fiche des métiers mobilisés



Étape 3 — Décrire les couches de défense en profondeur

docker-compose.yml | rapport-audit.md | cves.csv

rapport-audit.md > # Audit de securite - Plateforme Code un max > ## 6. Approches, circuit d'un incident

```
# Audit de securite - Plateforme Code un max

## 1. Perimetre

Cible : dvwa-cible 10.40.0.10, clone de la plateforme de cours.

Poste d'audit : attaquant 10.40.0.20.

Reseau : cyberlab-net 10.40.0.0/16.

## 2. Roles mobilises

- Analyste SOC : surveillance et detection sur le reseau du lab.

- Pentester : recherche active de vulnerabilites sur dvwa-cible.

- Ingenieur Securite Cloud : durcissement de l'hebergement.

- Consultant Cybersecurite : synthese et recommandations.

- RSSI : arbitrage des risques et validation du plan.

- Forensic Expert : analyse post-incident.


## 3. Approches

- Blue Team : defense, detection, reaction.

- Red Team : simulation d'attaque offensive.

## 4. Defense en profondeur : couches

- réseau : segmentation via cyberlab-net, filtrage entre hotes

- hôte : surface d'attaque du conteneur dvwa-cible

- application : controles d'entree de la plateforme web

- données : protection des informations de cours

- détection : collecte des traces pour le SOC

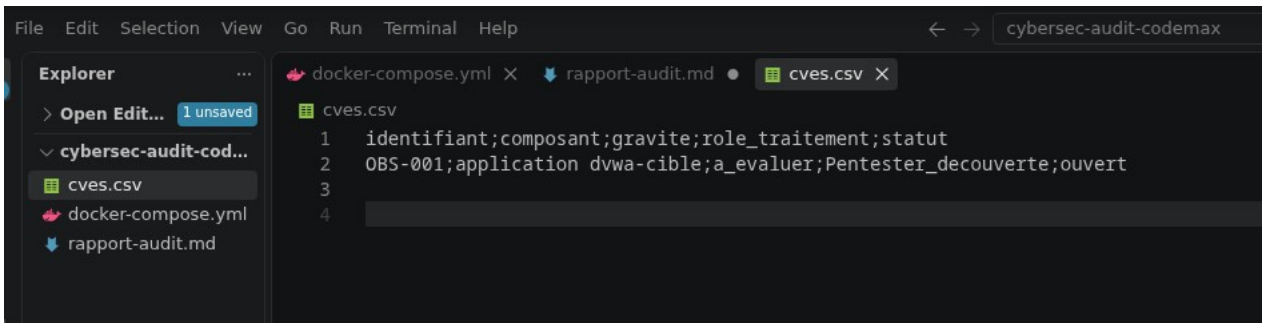
```

Étape 4 — Vérifier la connectivité du périmètre d'audit

```
anto@debian:~/cybersec-audit-codemax$ docker exec attaquant ping -c 10 10.40.0.10
PING 10.40.0.10 (10.40.0.10) 56(84) bytes of data.
64 bytes from 10.40.0.10: icmp_seq=1 ttl=64 time=0.129 ms
64 bytes from 10.40.0.10: icmp_seq=2 ttl=64 time=0.066 ms
64 bytes from 10.40.0.10: icmp_seq=3 ttl=64 time=0.069 ms
64 bytes from 10.40.0.10: icmp_seq=4 ttl=64 time=0.090 ms
64 bytes from 10.40.0.10: icmp_seq=5 ttl=64 time=0.086 ms
64 bytes from 10.40.0.10: icmp_seq=6 ttl=64 time=0.068 ms
64 bytes from 10.40.0.10: icmp_seq=7 ttl=64 time=0.050 ms
64 bytes from 10.40.0.10: icmp_seq=8 ttl=64 time=0.084 ms
64 bytes from 10.40.0.10: icmp_seq=9 ttl=64 time=0.088 ms
64 bytes from 10.40.0.10: icmp_seq=10 ttl=64 time=0.177 ms

--- 10.40.0.10 ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 9222ms
rtt min/avg/max/mdev = 0.050/0.090/0.177/0.035 ms
anto@debian:~/cybersec-audit-codemax$
```

Étape 5 — Initialiser le registre des vulnérabilités



Étape 6 — Formaliser le circuit de gestion d'incident et l'articulation Red/Blue

```
## 4. Defense en profondeur : couches

- réseau : segmentation via cyberlab-net, filtrage entre hotes

- hôte : surface d'attaque du conteneur dvwa-cible

- application : controles d'entree de la plateforme web

- données : protection des informations de cours

- détection : collecte des traces pour le SOC

## 5. Constats et risques

### Analyste SOC (Security Opérations Center)

Depuis le poste d'audit, confirmation que la cible est joignable sur le réseau du lab.
Un ping limité à 10 paquets est lancé depuis le conteneur attaquant vers 10.40.0.10.
docker exec attaquant bash -c "apt-get update && apt-get install -y iputils-ping curl"
docker exec attaquant ping -c 10 10.40.0.10
Résultat attendu : Le ping aboutit avec 0% de perte

### Analyse des risques et gestion des vulnérabilités

Création du tableau CSV cves.csv avec les colonnes identifiant, composant, gravité, rôle de traitement et statut.
Première ligne d'observation reliée au Pentester OBS-001 avec un statut ouvert.

## 6. Approches, circuit d'un incident

- Blue Team : defense, detection, reaction

- Red Team : simulation d'attaque offensive
```

Exercice 03 — Dossier de reconnaissance de laplateforme Code un max

Étape 1 — Confirmer la joignabilité de la cible

```
anto@debian:~/cybersec-audit-codemax$ docker exec attaquant ping -c 10 10.40.0.10
PING 10.40.0.10 (10.40.0.10) 56(84) bytes of data.
64 bytes from 10.40.0.10: icmp_seq=1 ttl=64 time=0.129 ms
64 bytes from 10.40.0.10: icmp_seq=2 ttl=64 time=0.066 ms
64 bytes from 10.40.0.10: icmp_seq=3 ttl=64 time=0.069 ms
64 bytes from 10.40.0.10: icmp_seq=4 ttl=64 time=0.090 ms
64 bytes from 10.40.0.10: icmp_seq=5 ttl=64 time=0.086 ms
64 bytes from 10.40.0.10: icmp_seq=6 ttl=64 time=0.068 ms
64 bytes from 10.40.0.10: icmp_seq=7 ttl=64 time=0.050 ms
64 bytes from 10.40.0.10: icmp_seq=8 ttl=64 time=0.084 ms
64 bytes from 10.40.0.10: icmp_seq=9 ttl=64 time=0.088 ms
64 bytes from 10.40.0.10: icmp_seq=10 ttl=64 time=0.177 ms

--- 10.40.0.10 ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 9222ms
rtt min/avg/max/mdev = 0.050/0.090/0.177/0.035 ms
anto@debian:~/cybersec-audit-codemax$
```

Étape 2 — Installer les outils offensifs manquants

```
anto@debian:~/cybersec-audit-codemax$ docker exec attaquant bash -c "apt-get update && apt-get install -y nmap"
Hit:1 http://deb.debian.org/debian bookworm InRelease
Hit:2 http://deb.debian.org/debian bookworm-updates InRelease
Hit:3 http://deb.debian.org/debian-security bookworm-security InRelease
Reading package lists...
Reading package lists...
Building dependency tree...
Reading state information...
The following additional packages will be installed:
  dbus dbus-bin dbus-daemon dbus-session-bus-common dbus-system-bus-common
  libapparmor1 libblas3 libdbus-1-3 libexpat1 liblinear4 liblua5.3-0
  libpcap0.8 libpcre3 lua-lpeg nmap-common
Suggested packages:
  default-dbus-session-bus | dbus-session-bus liblinear-tools liblinear-dev
  ncat ndiff zenmap
The following NEW packages will be installed:
  dbus dbus-bin dbus-daemon dbus-session-bus-common dbus-system-bus-common
  libapparmor1 libblas3 libdbus-1-3 libexpat1 liblinear4 liblua5.3-0
  libpcap0.8 libpcre3 lua-lpeg nmap nmap-common
0 upgraded, 16 newly installed, 0 to remove and 1 not upgraded.
Need to get 7791 kB of archives.
After this operation, 30.7 MB of additional disk space will be used.
Get:1 http://deb.debian.org/debian bookworm/main amd64 libdbus-1-3 amd64 1.14.10-1~deb12u1 [201 kB]
Get:2 http://deb.debian.org/debian bookworm/main amd64 dbus-bin amd64 1.14.10-1~deb12u1 [105 kB]
Get:3 http://deb.debian.org/debian bookworm/main amd64 dbus-session-bus-common all 1.14.10-1~deb12u1 [78.2 kB]
Get:4 http://deb.debian.org/debian bookworm/main amd64 libapparmor1 amd64 3.0.8-3 [41.2 kB]
Get:5 http://deb.debian.org/debian-security bookworm-security/main amd64 libexpat1 amd64 2.5.0-1+deb12u5 [108 kB]
```

```
anto@debian:~/cybersec-audit-codemax$ docker exec attaquant bash -c "DEBIAN_FRONTEND=noninteractive apt-get install -y tshark"
Reading package lists...
Building dependency tree...
Reading state information...
The following additional packages will be installed:
  libbcg729-0 libc-ares2 libdeflate0 libglib2.0-0 libglib2.0-data libicu72
  libjbig0 libjpeg62-turbo liblerc4 liblua5.2-0 libmaxminddb0 libnl-3-200
  libnl-genl-3-200 libsbc1 libsmi2ldbl libsnappy1v5 libspandsp2 libspeexdsp1
  libssh-gcrypt-4 libtiff6 libwebp7 libwireshark-data libwireshark16
  libwiretap13 libwsutil14 libxml2 shared-mime-info wireshark-common
  xdg-user-dirs
Suggested packages:
  low-memory-monitor mmdb-bin snmp-mibs-downloader geoipupdate geoip-database
  geoip-database-extra libjs-leaflet libjs-leaflet.markercluster wireshark-doc
The following NEW packages will be installed:
  libbcg729-0 libc-ares2 libdeflate0 libglib2.0-0 libglib2.0-data libicu72
  libjbig0 libjpeg62-turbo liblerc4 liblua5.2-0 libmaxminddb0 libnl-3-200
  libnl-genl-3-200 libsbc1 libsmi2ldbl libsnappy1v5 libspandsp2 libspeexdsp1
  libssh-gcrypt-4 libtiff6 libwebp7 libwireshark-data libwireshark16
  libwiretap13 libwsutil14 libxml2 shared-mime-info tshark wireshark-common
  xdg-user-dirs
0 upgraded, 30 newly installed, 0 to remove and 1 not upgraded.
Need to get 36.0 MB of archives.
After this operation, 189 MB of additional disk space will be used.
Get:1 http://deb.debian.org/debian bookworm/main amd64 libbcg729-0 amd64 1.1.1-2 [33.1 kB]
Get:2 http://deb.debian.org/debian bookworm/main amd64 libc-ares2 amd64 1.18.1-3 [102 kB]
Get:3 http://deb.debian.org/debian bookworm/main amd64 libdeflate0 amd64 1.14-1 [61.4 kB]
Get:4 http://deb.debian.org/debian bookworm/main amd64 libglib2.0-0 amd64 2.74.6-2+deb12u9 [1403 kB]
```

Étape 3 — Cartographier les services exposés

```
anto@debian:~/cybersec-audit-codemax$ docker exec attaquant nmap -p- 10.40.0.10
Starting Nmap 7.93 ( https://nmap.org ) at 2026-10-27 11:34 UTC
Nmap scan report for dvwa-cible.cybersec-audit-codemax_cyberlab-net (10.40.0.10)
Host is up (0.000011s latency).
Not shown: 65534 closed tcp ports (reset)
PORT   STATE SERVICE
80/tcp open  http
MAC Address: 7E:F4:84:5A:13:51 (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 2.04 seconds
```

```
anto@debian:~/cybersec-audit-codemax$ docker exec attaquant nmap -sV -p 80 10.40.0.10
Starting Nmap 7.93 ( https://nmap.org ) at 2026-10-27 11:37 UTC
Nmap scan report for dvwa-cible.cybersec-audit-codemax_cyberlab-net (10.40.0.10)
Host is up (0.000087s latency).

PORT   STATE SERVICE VERSION
80/tcp open  http    Apache httpd 2.4.25 ((Debian))
MAC Address: 7E:F4:84:5A:13:51 (Unknown)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 7.02 seconds
```

Étape 4 — Réaliser le fingerprinting de l application web

```
anto@debian:~/cybersec-audit-codemax$ docker exec attaquant curl -s -I http://10.40.0.10/
HTTP/1.1 302 Found
Date: Tue, 27 Oct 2026 11:43:44 GMT
Server: Apache/2.4.25 (Debian)
Set-Cookie: PHPSESSID=9vuo05kjsq4pje98kg31iiar62; path=/
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate
Pragma: no-cache
Set-Cookie: PHPSESSID=9vuo05kjsq4pje98kg31iiar62; path=/
Set-Cookie: security=low
Location: login.php
Content-Type: text/html; charset=UTF-8
```

Étape 5 — Capturer et analyser le trafic HTTP

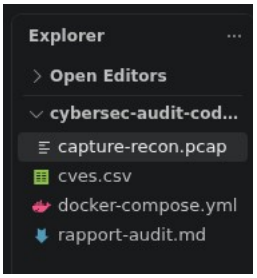
```
anto@debian:~/cybersec-audit-codemax$ docker exec -it attaquant bash
root@cfe3cc9e97da:/# ls
bin  boot  data  dev  etc  home  lib  lib64  media  mnt  opt  proc  root  run  sbin  srv  sys  tmp  usr  var
root@cfe3cc9e97da:/# chmod 777 -R /data
```

```
anto@debian:~/cybersec-audit-codemax$ sudo docker exec attaquant bash -c "(sleep 2; for i in 1 2 3 4 5 6; do curl -s -o /dev/null http://10.40
.0.10/; done) & tshark -i eth0 -f 'host 10.40.0.10 and port 80' -c 10 -a duration:20 -w /data/capture-recon.pcap"
Running as user "root" and group "root". This could be dangerous.
Capturing on 'eth0'
 ** (tshark:4088) 13:11:21.889546 [Main MESSAGE] -- Capture started.
 ** (tshark:4088) 13:11:21.889697 [Main MESSAGE] -- File: "/data/capture-recon.pcap"
tshark:
10 packets captured
anto@debian:~/cybersec-audit-codemax$ ^C
anto@debian:~/cybersec-audit-codemax$ ^C
anto@debian:~/cybersec-audit-codemax$ docker exec attaquant ls -l /data/capture-recon.pcap
-rw------- 1 root root 1904 Oct 27 13:11 /data/capture-recon.pcap
```

Étape 6 — Manipuler une requête avec les outils d interception

```
anto@debian:~/cybersec-audit-codemax$ docker exec attaquant curl -s -v -H "User-Agent: AuditCodeMax" http://10.40.0.10/login.php -o /dev/null
*   Trying 10.40.0.10:80...
* Connected to 10.40.0.10 (10.40.0.10) port 80 (#0)
> GET /login.php HTTP/1.1
> Host: 10.40.0.10
> Accept: */*
> User-Agent: AuditCodeMax
>
< HTTP/1.1 200 OK
< Date: Tue, 27 Oct 2026 11:51:49 GMT
< Server: Apache/2.4.25 (Debian)
< Set-Cookie: PHPSESSID=j2cgflprb92lgq6p18274ucrd6; path=/
< Expires: Tue, 23 Jun 2009 12:00:00 GMT
< Cache-Control: no-cache, must-revalidate
< Pragma: no-cache
< Set-Cookie: PHPSESSID=j2cgflprb92lgq6p18274ucrd6; path=/
< Set-Cookie: security=low
< Vary: Accept-Encoding
< Content-Length: 1523
< Content-Type: text/html;charset=utf-8
<
{ [1523 bytes data]
* Connection #0 to host 10.40.0.10 left intact
```

Étape 7 — Rédiger le dossier de reconnaissance



7. Methodologie de reconnaissance

Confirmer la joignabilité de la cible avec PING

docker exec attaquant ping -c 10 10.40.0.10

8. Les outils offensifs : curl nmap et Wireshark / tshark - Reconnaissance du 27/10/2026

- Installer nmap et tshark

docker exec attaquant bash -c "apt-get update && apt-get install -y nmap"

docker exec attaquant bash -c "DEBIAN_FRONTEND=noninteractive apt-get install -y tshark"

- Scanner les ports ouverts de la cible

docker exec attaquant nmap -p- 10.40.0.10

-> le port 80 est exposé

- Détecter les services et leurs versions

docker exec attaquant nmap -sV -p 80 10.40.0.10

On révèle le nom du serveur et sa version exacte :

Apache httpd 2.4.25 ((Debian))

- Réaliser le fingerprinting de l application web avec curl

docker exec attaquant curl -s -I http://10.40.0.10/

L'en-tête Server révèle Apache, le cookie PHPSESSID trahit PHP, et la redirection vers login.php confirme uneapplication avec authentification.

- Interrogation de la page de connexion

docker exec attaquant curl -s -v -H "User-Agent: AuditCodeMax" http://10.40.0.10/login.php -o /dev/null

-> Fiche de fingerprinting : serveur Apache, application PHP, présence dun cookie de session et redirection vers login.php.

- Wireshark / tshark : capture et analyse de trafic

Ouverture des permission dur le dossier /data de l'instance "attaquant"

docker exec -it attaquant bash

chmod 777 -R /data

tshark lancé en capture limitée à quelques paquets pendant qu'une requête vers DVWA est émise, pour observer le dialogue HTTP :

docker exec attaquant bash -c "(sleep 2; for i in 1 2 3 4 5 6; do curl -s -o /dev/null http://10.40.0.10/; done) & tshark -i eth0 -f 'host 10.40.0.10 and port 80' -c 10 -a duration:20 -w /data/capture-recon.pcap"

Le filtre de capture -f restreint aux paquets échangés avec la cible sur le port 80 ; le fichier .pcap est écrit dans/data.

10 paquets sont capturés, et bien enregistrés dans data/capture-recon.pcap :

docker exec attaquant ls -l /data/capture-recon.pcap